

Views need not even be only about generating HTML content. You may also want to provide an API for your website that allows developers to access your data as JSON or XML. Or you might want users to be able to export their personal data as a CSV file. Another possibility could be to allow users to save articles in PDF format.

Django helps you develop views for all these use cases and more.

Templates

Django includes its own templating language that can be used to generate HTML code for your web pages.

With Django's template language you can write your frontend HTML code as normal, and simply use special template block that designate where data need to be filled in. Then in your view functions you can combine this template with the required data and return that as a response to display in the browser.

The Django template language is quite simple, but still quite powerful and extensible. One of the most useful features of this language is the ability to extend one template from another, and to import common template code.

For instance you can create a base template for every page in your website to follow. This template can contain the basic JavaScript and CSS code for your website. You can then extend this template to create a base template for all your article pages, another base template for all your gallery pages, etc. These can further be extended based on your need.

We also mentioned how it is possible to extend Django the template language. In fact Django itself ships with some convenient extensions that can be enabled if needed. For instance Django ships with an app called `humanize` that simplifies conversions such as formatting numbers with commas as separators, converting numbers like '4500000' to '4.5 million', converting absolute dates to relative dates etc.

The template language used in Django is designed to have minimal logic, and no complex code. Any processing you need to do should happen in the view and the template should merely get the final data it needs to display.

As of Django 1.8 it is simpler to plug in other third-party template languages in Django; however since the view has never been restricted from using other third-party code it was always possible to use any Python-based template language to generate your responses.

Forms

Often times you need to get just a few details from a user, such as their